

Auditoria da Camada de API

Quanta Shop — .NET Core API

Data: 20/03/2026 | Análise de Segurança, Qualidade e Escalabilidade

Dimensão da API

	V1	V2
Controllers	32	14
Endpoints	~240	~63
Total	272	77

Uma API de escala real. O problema é que 32 controllers em V1 nunca foram migrados para V2, e os dois vivem lado a lado sem estratégia de versioning definida.

Segurança — Os Problemas Mais Graves

1. CORS completamente aberto

`SetIsOriginAllowed(host => true)` — qualquer domínio pode chamar a API. Em produção, qualquer site externo pode fazer chamadas autenticadas. Deveria ter `allowlist` de domínios.

2. JWT com validação enfraquecida

`ValidateIssuer = false` e `ValidateAudience = false` desativados. Token não verifica origem nem destino. Em um sistema financeiro com cashback e saques, isso é superfície de ataque desnecessária.

3. Token com validade de 24 horas

Sessão muito longa. Se um token for comprometido, atacante tem janela grande de ação. Padrão recomendado: 15-60 minutos com refresh token.

4. Nenhum rate limiting em nenhum endpoint

Endpoints de login, cadastro e recuperação completamente expostos. Sem proteção contra força bruta e enumeração de usuários.

5. Cookie de debug expõe stack traces em produção

Cookie `"oh_vida_oh_ceus"` força erros e retorna stack trace completo — expõe nomes de classes, estrutura do banco, e caminhos internos.

6. `ZanoxConnectId` hardcoded no código

ID de credencial de afiliado literal no `LoggedControllerBase` — herdado por praticamente todos os controllers da V1.

1. Sync-over-async espalhado	.Result/.Wait() em métodos async criam risco de deadlock. API pode travar sob carga.
2. Fire-and-forget sem tratamento	_ = EnviarEmailConfirmacao(...) — erro é descartado. Sem log, sem retry, sem alerta.
3. async void em background service	AtualizaDadosAfilio.cs usa async override void TimerTick. Exceção derruba processo inteiro.
4. N+1 queries em controllers	foreach + query dentro = 100 credenciamentos = 100 queries. Padrão correto não foi aplicado globalmente.
5. Controllers com lógica pesada	UsuarioController: 31 endpoints com validação manual, serviços externos, formatação. Tudo em um lugar.
6. Respostas inconsistentes	DTOs tipados, objetos anônimos, strings brutas. Sem envelope padronizado. Impossibilita SDK futuro.
7. Acessar variável de ambiente no controller	Environment.GetEnvironmentVariable("SENDGRID_API_KEY") diretamente. Deve ser injetado via IOption<>.

Matriz de Qualidade

Aspecto	Nota
Escopo e cobertura de endpoints	8/10
Segurança (CORS, JWT, rate limit)	3/10
Qualidade do código assíncrono	3/10
Consistência das respostas	4/10
Separação de responsabilidades	4/10
Observabilidade (logging)	3/10

Prioridades Imediatas

- Ø=Ý4 Rate limiting nos endpoints de auth (Brute force risk)
- Ø=Ý4 Restringir CORS para domínios autorizados (Qualquer site pode chamar)
- Ø=Ý4 Remover/proteger cookie de debug (Exposição de internals)
- Ø=Ý4 Reduzir JWT validade + ValidateIssuer (Janela de ataque)
- Ø=ßá Eliminar .Result/.Wait() (Deadlock sob carga)
- Ø=ßá Padronizar envelope de resposta (Quebras no frontend)
- Ø=ßá Logging estruturado (Zero visibilidade em produção)

Análise gerada a partir de inspeção completa do código da
API .NET Core.
Total de controllers auditados: 46 | Endpoints analisados: 300+